

HHBKTendo

Spielesammlung – Pflichtenheft

HHBK Düsseldorf · Lernfeld 5

HHBKTendo Research Center · 2026

Strategiespiele mit MiniMax-KI (Bauernschach & Tic-Tac-Toe)

Version 1.5

Pflichtenheft – HHBKTendo Spielesammlung

Projekt: Strategiespiele-Prototyp mit MiniMax-KI

Auftraggeber: HHBK Tendo Research Center

Auftragnehmer: Projektteam (Lernfeld 5)

Version: 1.4

Datum: 2026-04-22

Status: Entwurf

Inhaltsverzeichnis

1. [Zielbestimmung](#)
 2. [Produkteinsatz](#)
 3. [Systemübersicht und Architektur](#)
 4. [Funktionale Anforderungen \(MUSS\)](#)
 5. [Funktionale Anforderungen \(SOLL / KANN\)](#)
 6. [Nicht-funktionale Anforderungen](#)
 7. [Datenmodell](#)
 8. [Benutzerschnittstelle](#)
 9. [Globale Variablen und Zustandsmodell](#)
 10. [MiniMax-Algorithmus und Bewertungsfunktionen](#)
 11. [Testfälle und Abnahmekriterien](#)
 12. [Dokumentations- und Konzeptanforderungen](#)
 13. [Liefergegenstände](#)
 14. [Projektplanung](#)
 15. [Design und Corporate Identity](#)
-

1 Zielbestimmung

1.1 Mussziele

ID	Herkunft (LH)	Beschreibung
PF-M01	LF4000	Mindestens zwei Strategiespiele sind vollständig spielbar.
PF-M02	LF4010	Alle Spiele sind in einer einzigen Python-Anwendung mit grafischer Benutzeroberfläche (tkinter) integriert.
PF-M03	LF4020	Jedes Spiel verwendet ein 6×6-Spielfeld.
PF-M04	LF4030	Die KI nutzt den MiniMax-Algorithmus für alle Spiele.
PF-M05	LF4040	Der menschliche Spieler macht immer den ersten Zug; die KI antwortet als zweiter Spieler.
PF-M06	LF4050	Für jedes Spiel ist die Spielstärke (Suchtiefe 1–5) vor Spielbeginn einstellbar.
PF-M07	LF4060	Ein laufendes Spiel kann jederzeit abgebrochen werden (mit Bestätigungsdialog).
PF-M08	LF4070	Benutzer können sich registrieren und anmelden.
PF-M09	LF4080	Siege und Niederlagen werden je Spiel und Schwierigkeitsgrad in der Datenbank gespeichert.
PF-M10	LF4090	Im Gastmodus werden keine Ergebnisse gespeichert.
PF-M11	LF4100	Die Bestenliste ist pro Spiel und Schwierigkeitsgrad separat abrufbar.
PF-M12	LF4110	Die Spielregeln jedes Spiels sind während des Spiels per Schaltfläche einsehbar.

1.2 Sollziele

ID	Herkunft (LH)	Beschreibung
PF-S01	LF4120	Alpha-Beta-Pruning optimiert den MiniMax-Algorithmus.
PF-S02	LD4220	Die Bestenliste wird durch SQL-Abfragen aus der Datenbank generiert.

1.3 Kannziele

ID	Herkunft (LH)	Beschreibung
PF-K01	LF4130	Der Benutzer kann die Sprache zwischen Englisch (Standard) und Deutsch umschalten.
PF-K02	LD4230	Spracheinstellung wird benutzerspezifisch in der Datenbank gespeichert.
PF-K03	LF4000	Das Spiel Dame kann als drittes Strategiespiel implementiert werden (vereinfachte Regeln, 6x6, MiniMax-KI).
PF-K04	LF4120	Eine definierte Schnittstelle ermöglicht die Einbindung einer alternativen KI anstelle von MiniMax.

1.4 Abgrenzung (Wont-have)

- Kein Netzwerkbetrieb / Multiplayer (LF4140)

2 Produkteinsatz

2.1 Anwendungsbereich

Freizeitanwendung zum Spielen von Strategiespielen gegen eine KI. Primärzweck: Markttest des MiniMax-Algorithmus und der Spielmechaniken.

2.2 Zielgruppe

Testpersonen im Alter 12–99, die Strategiespiele bevorzugen, aus den Märkten: Deutschland, UK, Singapur, USA, Hongkong.

2.3 Produktumgebung

Eigenschaft	Wert
Betriebssystem	Windows 10 / 11, macOS
Programmiersprache	Python 3.10+
GUI-Framework	tkinter (stdlib)
Datenbank	SQLite (unabhängig, dateibasiert)
Programmierparadigma	Prozedural, funktionsorientiert (keine Klassen für Spiellogik)

3 Systemübersicht und Architektur

3.1 Modulübersicht

```
HHBKTendo Spielesammlung
|
|— main.py          GUI-Steuerung (Login, Menü, Spielscreen, Bestenliste)
|— auth.py          Authentifizierung (Registrierung, Login, Session)
|— database.py       Datenbankoperationen (SQLite, CRUD)
|— minimax.py       Generischer MiniMax + Alpha-Beta-Pruning (spielunabhängig)
|— pawn_chess.py    Spiellogik Bauernschach (Brett, Züge, Bewertung)
|— tictactoe.py     Spiellogik Tic-Tac-Toe 4-gewinnt (6×6, 4 in einer Reihe, Brett,
Züge, Bewertung)
|— test_all.py      Tests, welche die Methoden nach Richtigkeit prüfen (erwartetes
Resultat wird geprüft)
|— games.db         SQLite-Datenbankdatei (auto-generiert beim Start)
```

3.2 Schnittstellen zwischen Modulen

Der MiniMax-Algorithmus ist vollständig spielunabhängig. Er erhält spielspezifische Callback-Funktionen:

```
minimax.get_best_move(
    board,          ← aktuelles Spielfeld (2D-Liste)
    depth,          ← Suchtiefe (= Schwierigkeit)
    get_moves_fn,   ← fn(board, is_maximizing) → [Züge]
    apply_move_fn,  ← fn(board, move, is_maximizing) → neues Board
    evaluate_fn,    ← fn(board) → int (positiv = gut für KI)
    is_terminal_fn  ← fn(board) → (bool, int)
)
```

Jedes Spielmodul implementiert exakt diese vier Funktionen. Damit ist **dieselbe KI-Engine** für alle Spiele wiederverwendbar (LN5040).

3.3 Datenfluss

```
Benutzerinteraktion (Klick)
→ on_board_click()
→ handle_*_click()
→ Spiellogikmodul (apply_move)
→ draw_board()
→ start_ai_turn() [Thread]
    → minimax.get_best_move()
        → Spiellogikmodul (get_valid_moves, apply_move, evaluate, is_terminal)
    → after_ai_turn() [Hauptthread via .after(0, ...)]
→ end_game() → database.save_result()
```

4 Funktionale Anforderungen (MUSS)

4.1 Spiel: Bauernschach (PF-M01)

Brettkonfiguration:

- 6×6 Felder, weiße Bauern (Mensch) in Reihe 5, schwarze Bauern (KI) in Reihe 0
- Repräsentation: `board[row][col]` $\in \{\text{WHITE}=1, \text{BLACK}=-1, \text{EMPTY}=0\}$

Erlaubte Züge:

Zugart	Bedingung
Vorwärtzug	Zielfeld ist leer (direkt vor dem Bauern in Richtung gegnerische Grundlinie)
Diagonales Schlagen	Zielfeld enthält gegnerischen Bauern (diagonal vorwärts)

Gewinnbedingungen:

Zustand	Ergebnis
Weißer Bauer erreicht Reihe 0	Mensch gewinnt
Schwarzer Bauer erreicht Reihe 5	KI gewinnt
Weiß hat keine Figuren mehr	KI gewinnt
Schwarz hat keine Figuren mehr	Mensch gewinnt
Aktueller Spieler hat keine gültigen Züge	Gegner gewinnt

Unentschieden ist nicht möglich.

Bewertungsfunktion:

```
score = E (Figurwert + Fortschritt × 2 + Zentrumbonus) für KI-Figuren
        - E (Figurwert + Fortschritt × 2 + Zentrumbonus) für Menschen-Figuren
```

- Figurwert: 10 Punkte pro Figur
- Fortschritt: Anzahl der Reihen, die eine Figur vorgerückt ist
- Zentrumbonus: +1 für Spalten 1–4

4.2 Spiel: Tic-Tac-Toe (4 gewinnt) (PF-M01)

Brettkonfiguration:

- 6×6 Felder, leer zu Spielbeginn
- Repräsentation: `board[row][col]` ∈ {HUMAN=1, AI=-1, EMPTY=0}

Spielregeln:

- Spieler setzen abwechselnd einen Stein auf ein leeres Feld
- Mensch beginnt (X), KI antwortet (O)

Gewinnbedingungen:

Zustand	Ergebnis
4 in einer Reihe (horizontal)	Erster Spieler gewinnt
4 in einer Spalte (vertikal)	Erster Spieler gewinnt
4 diagonal	Erster Spieler gewinnt
Brett voll (36 Felder), kein Gewinner	Unentschieden

Bewertungsfunktion:

```
score = (3er-Linien KI) × 50 - (3er-Linien Mensch) × 50  
       + (2er-Linien KI) × 10 - (2er-Linien Mensch) × 10  
       + Zentrumsnähe-Bonus
```

Nur offene Linien (ohne gegnerische Steine im Fenster) werden gezählt.

4.3 KI-System (PF-M04)

MiniMax mit Alpha-Beta-Pruning:

```
minimax(board, depth, is_maximizing, α=-∞, β=+∞):  
    if terminal(board): return terminal_score  
    if depth == 0:      return evaluate(board)  
  
    if is_maximizing:  
        best = -∞  
        for move in get_moves(board, True):  
            score = minimax(apply(board, move), depth-1, False, α, β)  
            best = max(best, score); α = max(α, score)  
            if β ≤ α: break # Beta-Pruning  
        return best  
    else:  
        best = +∞  
        for move in get_moves(board, False):  
            score = minimax(apply(board, move), depth-1, True, α, β)  
            best = min(best, score); β = min(β, score)  
            if β ≤ α: break # Alpha-Pruning  
        return best
```

Schwierigkeitsgrade:

Level	Suchtiefe	Verhalten
1 – Leicht	1	Nur einen Zug voraus, kaum strategisch
2 – Mittel	2	Einfache Taktiken erkannt
3 – Schwer	3	Mittlere Planung (Standard)
4 – Experte	4	Starke KI, schwer zu schlagen
5 – Meister	5	Sehr starke KI, KI-Zug kann mehrere Sekunden dauern

Zeitlimit: KI-Zug läuft in eigenem Thread (max. 45 Sekunden, LN5010).

4.4 Benutzerverwaltung (PF-M08-M10)

Funktion	Beschreibung
Registrierung	Benutzername (min. 3 Zeichen), Passwort (min. 4 Zeichen), Eindeutigkeit geprüft
Login	Benutzername + Passwort, Verifikation gegen gespeicherten Hash
Gastmodus	Spiel ohne Registrierung, keine Speicherung
Passwort-Hashing	SHA-256 + 16-Byte Zufalls-Salt, Format: <code>salt:hash</code>

4.5 Bestenliste (PF-M09, M11)

- Speicherung: User-ID, Spielname, Schwierigkeitsgrad, gewonnen (0/1)
 - Abfrage: gruppiert nach Benutzer, sortiert nach Siegen absteigend
 - Anzeige: Rang, Benutzername, Siege, Niederlagen, Gesamt-Spiele
 - Unentschieden (nur Tic-Tac-Toe) werden nicht gespeichert
-

5 Funktionale Anforderungen (SOLL / KANN)

5.1 Sprachumschaltung (PF-K01)

- Umschaltung zwischen EN (Standard) und DE jederzeit möglich
 - Alle UI-Texte, Regeln und Statusmeldungen werden in der gewählten Sprache angezeigt
 - Spracheinstellung wird bei eingeloggten Benutzern in der DB persistiert
-

6 Nicht-funktionale Anforderungen

ID	Anforderung	Maßnahme
NF-01 (LN5000)	Python + geeignetes GUI-Framework	Python 3.10+, tkinter
NF-02 (LN5001)	Prozedurale Programmierung	Keine Klassen für Spiellogik; Module mit Funktionen
NF-03 (LN5010)	KI-Zug max. 45 Sekunden	KI in separatem Thread
NF-04 (LN5020)	Persistente Benutzerdaten	SQLite-Datenbankdatei
NF-05 (LN5030)	Passwörter nicht im Klartext	SHA-256 + Salt (auth.py)
NF-06 (LN5040)	Wiederverwendbare Software	Generischer MiniMax, spielunabhängige Schnittstelle
NF-07 (LN5050)	Intuitive Bedienung	Einheitliches Design, klare Labels, Statusanzeige
NF-08 (LN5060)	Brand Identity	Dunkles Farbschema (HHBKTendo CI), konsistentes Layout
NF-09	Plattformkompatibilität	Buttons als <code>tk.Label</code> mit Bindings implementiert, da macOS den <code>bg / fg</code> -Parameter von <code>tk.Button</code> ignoriert

7 Datenmodell

7.1 Tabelle: `users`

Spalte	Typ	Beschreibung
id	INTEGER PK AUTO	Primärschlüssel
username	TEXT UNIQUE NOT NULL	Eindeutiger Benutzername
password_hash	TEXT NOT NULL	SHA-256-Hash mit Salt (Format: <code>salt:hash</code>)
language	TEXT DEFAULT 'en'	Sprachpräferenz
created_at	TIMESTAMP	Registrierungszeitpunkt

7.2 Tabelle: `results`

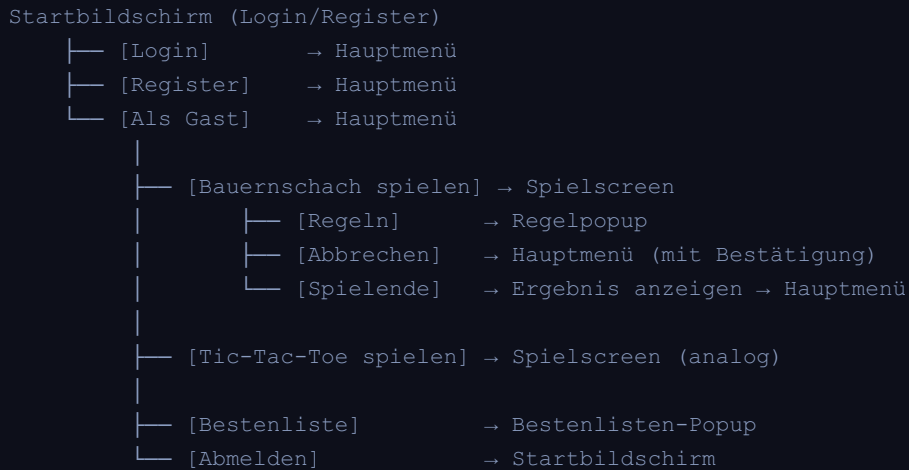
Spalte	Typ	Beschreibung
id	INTEGER PK AUTO	Primärschlüssel
user_id	INTEGER FK	Verweis auf <code>users.id</code>
game	TEXT	<code>'pawn_chess'</code> oder <code>'tictactoe'</code>
difficulty	INTEGER	Suchtiefe 1–5
won	INTEGER	1 = Sieg, 0 = Niederlage
played_at	TIMESTAMP	Zeitpunkt des Spielendes

7.3 Bestenlisten-Abfrage (SQL)

```
SELECT u.username,
       SUM(r.won)      AS wins,
       SUM(1 - r.won)  AS losses,
       COUNT(*)        AS total_games
FROM results r
JOIN users u ON r.user_id = u.id
WHERE r.game = ? AND r.difficulty = ?
GROUP BY r.user_id
ORDER BY wins DESC, losses ASC
LIMIT 10;
```

8 Benutzerschnittstelle

8.1 Bildschirmfluss



8.2 Spielfeld-Design

Bauernschach:

- Helles Feld: #eccc99 , Dunkles Feld: #8b5e3c
- Weiße Figur: weißer Kreis (#ffffff) mit ♔-Symbol, schwarzer Kontur
- Schwarze Figur: dunkler Kreis (#1a1a2e) mit ♚-Symbol, helle Kontur
- Ausgewählte Figur: rotes Feld, gültige Züge: grünes Feld

Tic-Tac-Toe (6x6):

- X (Mensch): hellblau (#64b5f6), dicke Linien
- O (KI): rot (#ef5350), dicke Ovallinie

8.3 Farbschema (CI)

Element	Farbe	Hex
Hintergrund dunkel	Nacht-Marine	#1a1a2e
Hintergrund Karten	Dunkel-Blau	#0f3460
Akzent (Buttons, Titel)	HHBKTendo-Rot	#e94560
Akzent2 (Sekundär)	Lila	#533483
Text (Standard)	Hellgrau	#eaeaea

9 Globale Variablen und Zustandsmodell

Das gesamte Laufzeit-Zustand der Anwendung ist im Dictionary `app_state` in `main.py` konzentriert (LN5001):

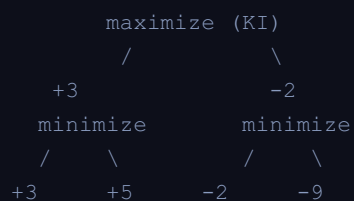
Variable	Typ	Bedeutung
<code>app_state["language"]</code>	str	Aktuelle Sprache ('en'/'de')
<code>app_state["current_screen"]</code>	tk.Frame	Aktuell angezeigter Screen
<code>app_state["game"]</code>	str	Aktives Spiel ('pawn_chess'/'tictactoe')
<code>app_state["board"]</code>	list[list[int]]	Aktuelles 6×6-Spielfeld
<code>app_state["difficulty"]</code>	int	Suchtiefe 1–5
<code>app_state["human_turn"]</code>	bool	True = Mensch am Zug
<code>app_state["selected"]</code>	tuple None	Ausgewählte Figur (Bauernschach)
<code>app_state["valid_moves"]</code>	list	Gültige Züge der ausgewählten Figur
<code>app_state["game_over"]</code>	bool	True nach Spielende
<code>app_state["ai_thinking"]</code>	bool	True während KI-Berechnung

In `auth.py`:

Variable	Typ	Bedeutung
<code>current_user</code>	dict None	Aktuell eingeloggter Benutzer (None = Gast)

10 MiniMax-Algorithmus und Bewertungsfunktionen

10.1 Illustrationsdiagramm



KI wählt den Zug, der zum Wert +3 führt (Minimizer wählt den schlechtesten für KI = +3 statt +5).

10.2 Alpha-Beta-Pruning

Äste des Spielbaums werden abgeschnitten, wenn feststeht, dass der Gegner diesen Pfad nie wählen wird:

- **Alpha (α)**: Bisher bestes Ergebnis für den Maximizer → wird nach oben weitergegeben
- **Beta (β)**: Bisher bestes Ergebnis für den Minimizer → wird nach unten weitergegeben
- Abbruch wenn $\beta \leq \alpha$

Einsparung: Im besten Fall $O(b^{(d/2)})$ statt $O(b^d)$ Knoten (b = Verzweigungsgrad, d = Tiefe).

10.3 Terminale Bewertungen

Zustand	Score
KI gewinnt	+1.000.000
Mensch gewinnt	-1.000.000
Unentschieden	0
Tiefenlimit (Blatt)	evaluate(board)

11 Testfälle und Abnahmekriterien

11.1 Funktionstests

TC-ID	Testfall	Erwartetes Ergebnis
TC-01	Registrierung mit gültigem Namen/Passwort	Benutzer angelegt, Login möglich
TC-02	Registrierung mit bereits vergebenem Namen	Fehlermeldung "Username already taken"
TC-03	Login mit falschem Passwort	Fehlermeldung "Incorrect password"
TC-04	Gastmodus – Spiel beenden	Kein Eintrag in Datenbank
TC-05	Bauernschach – Zug vorwärts auf leeres Feld	Bauer bewegt sich
TC-06	Bauernschach – Zug auf eigene Figur	Zug wird verweigert
TC-07	Bauernschach – Diagonales Schlagen	Gegnerischer Bauer wird entfernt
TC-08	Bauernschach – Bauer erreicht Grundlinie	Spiel endet mit korrektem Gewinner
TC-09	TTT – 4 in einer Reihe (horizontal)	Spiel endet, Gewinner korrekt
TC-10	TTT – 4 in einer Diagonale	Spiel endet, Gewinner korrekt
TC-11	TTT – Alle 36 Felder belegt ohne Gewinner	Unentschieden angezeigt
TC-12	KI-Zug bei Suchtiefe 5	KI zieht innerhalb 45 Sekunden
TC-13	Spielabbruch	Bestätigungsdialog, Rückkehr ins Hauptmenü
TC-14	Bestenliste anzeigen	Korrekte Sortierung nach Siegen
TC-15	Sprachwechsel EN → DE	Alle UI-Texte auf Deutsch

11.2 Abnahmekriterien (aus Lastenheft)

- Alle MUSS-Anforderungen (PF-M01 bis PF-M12) sind implementiert und testbar
- Live-Demo zeigt mind. 2 vollständige Spiele gegen KI
- Bestenliste zeigt korrekte Ergebnisse eingeloggter Benutzer
- Passwörter sind nicht im Klartext in der DB gespeichert (prüfbar per DB-Viewer)
- Anwendung startet auf Zielumgebung (Windows 10/11) mit `python main.py`

12 Dokumentations- und Konzeptanforderungen

12.1 Benutzerdokumentation

ID	Herkunft (LH)	Beschreibung
DOC-01	LD5100	Die Benutzerdokumentation ermöglicht einer nicht am Projekt beteiligten Person die eigenständige Installation und Ausführung der Anwendung.

12.2 Technische Dokumentation

ID	Herkunft (LH)	Beschreibung
DOC-02	LD5200	Beschreibung des Spielverhaltens zur Laufzeit (Zustandsübergänge, Spielablauf).
DOC-03	LD5210	Erläuterung des MiniMax-Algorithmus und der Bewertungsfunktionen je Spiel, inkl. Schwierigkeitsgrad.
DOC-04	LD5220	Beschreibung der Software-Architektur (Module, Schnittstellen, Datenfluss).
DOC-05	LD5230	Beschreibung aller verwendeten globalen Variablen und des zentralen <code>app_state</code> -Dictionaries.

12.3 Abschlusspräsentation

ID	Herkunft (LH)	Beschreibung
DOC-06	LD5300	SOLL/IST-Vergleich der realisierten Features (Lastenheft vs. Pflichtenheft vs. Umsetzung vs. Test).
DOC-07	LD5310	SOLL/IST-Vergleich der Projektplanung (Pflichtenheft-Zeitplan vs. tatsächlicher Projektverlauf).
DOC-08	LD5320	Darstellung wesentlicher Softwarekomponenten oder Ideen (z. B. MiniMax, Bewertungsfunktion, GUI).
DOC-09	LD5330	Aufführung verwendeter Quellen und Werkzeuge (Bibliotheken, Referenzen, KI-Tools).
DOC-10	LD5340	Fazit zum Projektverlauf: Zusammenarbeit, Aufwand, Lessons learnt.

12.4 Konzept Corporate Identity & Branding

ID	Herkunft (LH)	Beschreibung
DOC-11	LD5400	Das CI-Konzept begründet die gewählten Design-Elemente und enthält ein Mission und Vision Statement für die Marke HHBKTendo.
DOC-12	LD5410	Englischsprachiges Pitch-Konzept inkl. Skript für eine mündliche Präsentation mit überzeugender Argumentation für potenzielle Investoren und hohem Aufmerksamkeitswert.

12.5 Konzept Arbeitszeitgestaltung

ID	Herkunft (LH)	Beschreibung
DOC-13	LD5500	Das Konzept stellt Plan- und IST-Arbeitszeiten gegenüber, benennt Risiken und Herausforderungen und formuliert Empfehlungen für künftige Rahmenbedingungen der Projektarbeit bei HHBKTendo.

13 Liefergegenstände

#	Liefergegenstand	Verantwortlich
1	Pflichtenheft (dieses Dokument)	Team
2	Quellcode (alle .py-Dateien + games.db)	Entwickler
3	Benutzerdokumentation (Installation, Start, Spielanleitung) – Anforderungen DOC-01	Dokumentation
4	Technische Dokumentation (Spielverhalten, Algorithmen, Architektur, globale Variablen) – Anforderungen DOC-02 bis DOC-05	Entwickler
5	Testprotokoll (TC-01 bis TC-15)	Test
6	Abschlusspräsentation (SOLL/IST Features & Zeitplan, Softwarekomponenten, Quellen, Fazit) – Anforderungen DOC-06 bis DOC-10	Team
7	Konzept Corporate Identity & Branding (Design-Begründung, Mission/Vision) – Anforderung DOC-11	Design
8	Pitch-Deck (englischsprachig, Investoren-Argumentation) – Anforderung DOC-12	Team
9	Konzept Arbeitszeitgestaltung (Plan vs. IST, Risiken, Empfehlungen) – Anforderung DOC-13	Projektleitung

14 Projektplanung

14.1 Projektphasen (Wasserfallmodell)

Phase	Inhalt	Zeitraum
Analysephase	Lastenheft lesen, Pflichtenheft erstellen, WBS, Zeitplan, Ressourcenplan	Woche 1 (Vollzeit)
Design & Implementierung	Architektur, Datenbankdesign, Spiellogik, MiniMax, GUI	Woche 2 (Vollzeit)
Test	Testprotokoll, Bugfixes, Abnahmetests	Woche 2 (Mitte + Ende)
Dokumentation	Benutzerdokumentation, Technische Doku, Präsentation, Pitch, CI-Konzept	Woche 2 (Ende)
Abschluss	Live-Demo, Präsentation, Fachgespräch (ca. 20+10 min)	Woche 3 (letzter Tag)

14.2 Offene Punkte

#	Offener Punkt	Referenz	Status
1	Dame als drittes Spiel implementieren	PF-K03	Optional / zeitabhängig
2	Alternative KI-Schnittstelle definieren und implementieren	PF-K04	Optional / zeitabhängig
3	Testprotokoll-Dokument erstellen und ausfüllen	TC-01–TC-15	Ausstehend
4	Benutzerdokumentation schreiben	DOC-01	Ausstehend
5	Technische Dokumentation vervollständigen	DOC-02– DOC-05	Ausstehend
6	CI/Branding-Konzept (inkl. Mission/Vision) erstellen	DOC-11	Ausstehend
7	Englischsprachigen Pitch erarbeiten	DOC-12	Ausstehend
8	Konzept Arbeitszeitgestaltung ausarbeiten	DOC-13	Ausstehend

15 Design und Corporate Identity

15.1 Markenidentität (Brand Identity)

HHBKTendo positioniert sich als modernes, technologiegetriebenes Spielestudio mit einem klaren Fokus auf algorithmische KI. Das visuelle Erscheinungsbild soll diesen Anspruch transportieren: **präzise, dunkel, modern** – wie die Oberfläche eines High-End-Gaming-Setups. Die Farbwelt orientiert sich bewusst an professioneller Gaming-Ästhetik und hebt sich damit von den pastelligen UI-Konventionen klassischer Business-Software ab.

Mission: Strategiespiele mit echter KI-Herausforderung für ein internationales Publikum zugänglich machen.

Vision: HHBKTendo als führende Marke für algorithmisch gestützte Einzelspieler-Strategiespiele im B2C-Markt etablieren.

15.2 Farbpalette

Die gesamte Farbwelt folgt dem offiziellen **HHBKTendo Design System (Investor Edition)**. Jede Farbe hat eine definierte semantische Rolle und einen CSS-Token-Namen sowie eine korrespondierende Python-Variable.

CSS-Token	Python-Variable	Hex	Name	Rolle
<code>--bg-deep</code>	<code>BG_DEEP</code>	<code>#0d0f1a</code>	Deep Navy	App-Hintergrund aller Screens
<code>--bg-card</code>	<code>BG_CARD</code>	<code>#1a2040</code>	Card Navy	Karten, Container, Dialoge
<code>--bg-mid</code>	<code>BG_MID</code>	<code>#1e2850</code>	Mid Navy	Eingabefelder, Surfaces, Header
<code>--primary</code>	<code>PRIMARY</code>	<code>#e8365d</code>	Hot Pink	Primär-CTA, Logo, Danger-Aktionen
<code>--secondary</code>	<code>SECONDARY</code>	<code>#6c3fc5</code>	Royal Purple	Sekundäre Buttons (Register, Leaderboard)
<code>--accent</code>	<code>ACCENT</code>	<code>#00d4ff</code>	Cyber Cyan	Active States, Outlines, Hover
<code>--text-primary</code>	<code>TEXT_PRIMARY</code>	<code>#ffffff</code>	White	Primärtext, Labels
<code>--text-secondary</code>	<code>TEXT_SECONDARY</code>	<code>#8a9cc0</code>	Slate Blue	Sekundärtext, Statuszeilen
<code>--text-muted</code>	<code>TEXT_MUTED</code>	<code>#4a5680</code>	–	Dezenter Text, Hints, Platzhalter

Spielfeld-Farben

Token	Hex	Verwendung
<code>board_light</code>	<code>#eecc99</code>	Helle Felder Bauernschach – klassisches Schach-Beige
<code>board_dark</code>	<code>#8b5e3c</code>	Dunkle Felder Bauernschach – warmes Holzbraun
<code>ttt_x</code>	<code>#00d4ff</code>	X-Stein (Mensch) in Tic-Tac-Toe – orientiert an ACCENT (Cyber Cyan): kühl, präzise
<code>ttt_o</code>	<code>#e8365d</code>	O-Stein (KI) in Tic-Tac-Toe – orientiert an PRIMARY (Hot Pink): aggressiv, Gegner
<code>valid_move</code>	<code>#2e7d32</code>	Gültige Züge (Bauernschach) – Grün: „hier kann ich hin“
<code>highlight</code>	<code>#e8365d</code>	Ausgewählte Figur – PRIMARY Hot Pink: „das ist aktiv“

15.2.1 Darkmode - Funktion

Anforderungen

- Umschaltbar (Light/Dark)
- Speicherung der Auswahl
- Systemeinstellung berücksichtigen

15.3 Typografie

Das Design System definiert zwei Schriftfamilien mit unterschiedlichen Einsatzbereichen:

Display & Headings – Orbitron

Einsatz	Schriftart	Gewicht	Größe
Logo / Haupttitel (HHBKTendo)	Orbitron	900 (Black)	28 pt
Screen-Überschriften (H1)	Orbitron	700 (Bold)	16 pt
Karten-Titel, Spielname (H2)	Orbitron	400 (Regular)	13 pt

Body, Buttons & Labels – Exo 2

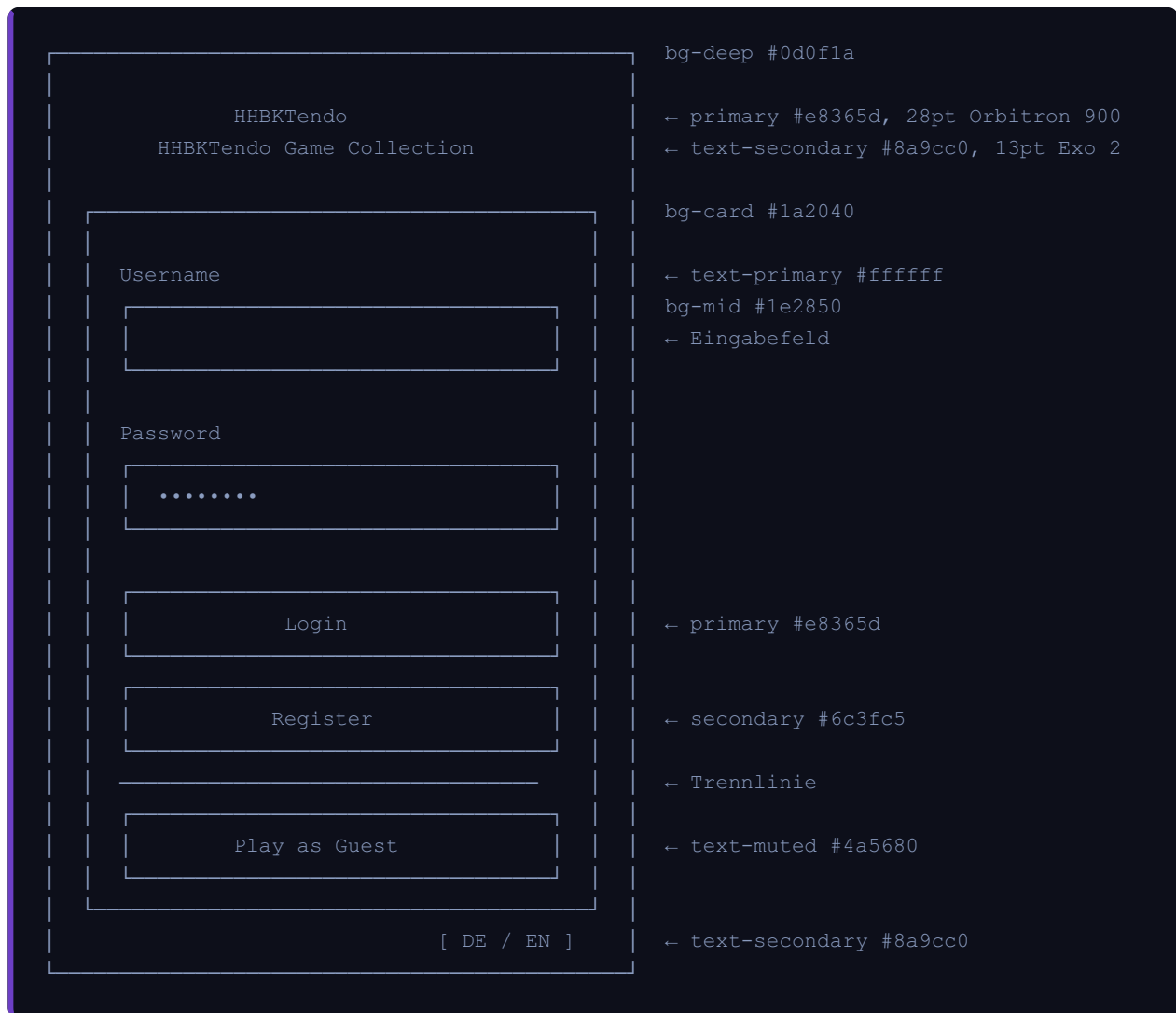
Einsatz	Schriftart	Gewicht	Größe
Buttons	Exo 2	700 (Bold)	11 pt
Labels, Fließtext	Exo 2	400 (Regular)	11–12 pt
Statuszeilen, Hints	Exo 2	300 (Light)	9–10 pt
Spielfigur-Symbole (Bauernschach)	Exo 2 / System	700 (Bold)	22 pt

Orbitron und Exo 2 sind als Google Fonts frei verfügbar. Im tkinter-Fallback (keine Netzwerkinstallation) werden sie durch `Courier` (Orbitron) bzw. `Helvetica` (Exo 2) ersetzt. Die Zielplattform Windows 10/11 unterstützt beide Schriften ohne Einschränkungen.

15.4 UI-Mockups

Die folgenden Mockups zeigen den finalen Screenfluss der Anwendung. Farbpräferenzen beziehen sich auf die Palette aus Abschnitt 15.2.

15.4.1 Login-Screen



15.4.2 Hauptmenü



15.4.3 Spielscreen – Bauernschach



15.4.4 Spielscreen - Tic-Tac-Toe (4 gewinnt)



15.4.5 Ergebnis-Overlay (Spielende)



15.4.6 Bestenliste (Popup)

Leaderboard - Pawn Chess - Hard

Rang	Spieler	Siege	Niederl
1	alice	7	2
2	bob	5	4
3	carol	3	1
4	dave	1	6

Close

bg-deep #0d0f1a

← primary #e8365d, Orbitron Bold

bg-card #1a2040

← text-primary #ffffff

← primary #e8365d

15.5 Design Tokens

Border Radius

Token	Wert	Verwendung
None	0 px	Spielfeld-Zellen, tabellarische Bereiche
Small	6 px	Buttons, Eingabefelder
Medium	12 px	Karten, Dialoge
Large	20 px	Panels, Overlays
Pill	100 px	Badge-Labels, Chip-Elemente

Spacing Scale

Token	Wert	Verwendung
XS	4 px	Interne Icon-Abstände
SM	8 px	Padding innerhalb von Buttons/Tags
MD	16 px	Standard-Padding in Karten
LG	24 px	Abstand zwischen Sektionen
XL	32 px	Große Außenabstände
XXL	48 px	Screen-seitliche Ränder

15.6 Glow & Effekte

Leuchtende Akzente (Glow) verstärken das Gaming-Feeling und heben interaktive Elemente hervor.

Effekt	CSS-Formel	Verwendung
Primary Glow	<code>box-shadow: 0 0 24px rgba(232, 54, 93, 0.35)</code>	Primär-Buttons (Login, Play, Close)
Secondary Glow	<code>box-shadow: 0 0 24px rgba(108, 63, 197, 0.35)</code>	Sekundär-Buttons (Register, Leaderboard)
Accent Glow	<code>box-shadow: 0 0 24px rgba(0, 212, 255, 0.35)</code>	Aktive Felder, Hover-Zustände

In tkinter werden Glow-Effekte durch Rahmenfarbe (`highlightbackground`) und leicht hellere Button-Farbe beim Hover simuliert.

15.7 Python Code-Export

Offizielle Token-Konstanten für die Implementierung in `main.py` :

```
# HHBKTendo Design System - Color Tokens
BG_DEEP      = "#0d0f1a"    # Deep Navy - App-Hintergrund
BG_CARD      = "#1a2040"    # Card Navy - Karten & Container
BG_MID       = "#1e2850"    # Mid Navy - Eingabefelder / Surfaces

PRIMARY      = "#e8365d"    # Hot Pink - Primär-CTA, Logo, Danger
SECONDARY    = "#6c3fc5"    # Royal Purple - Sekundäre Buttons
ACCENT       = "#00d4ff"    # Cyber Cyan - Active States, Outlines

TEXT_PRIMARY = "#ffffff"    # White - Primärtext
TEXT_SECONDARY = "#8a9cc0"  # Slate Blue - Sekundärtext
TEXT_MUTED   = "#4a5680"    # - Dezentertext, Hints
```

15.8 Designprinzipien

Prinzip	Umsetzung
Dunkel-First	Alle Hintergründe dunkel – kein heller Screen in der gesamten App
Farbhierarchie	Hot Pink (primary) = primär, Royal Purple (secondary) = sekundär, Slate Blue = passiv
Konsistenz	Alle interaktiven Elemente (Buttons) gleiche Form, Schrift und Padding
Feedback	Jeder Zustandswechsel hat eine visuelle Entsprechung (Farbe, Symbol)
Plattformkompatibilität	Buttons als <code>tk.Label</code> mit Bindings – funktioniert auf Windows und macOS
Spielfeld-Isolation	Brett-Farben (Beige/Braun) bewusst aus der CI-Palette herausgehalten, um Spielbereich klar abzugrenzen

Version	Datum	Änderung
1.0	2026-04-20	Erstversion
1.1	2026-04-21	Tic-Tac-Toe zwischenzeitlich auf 3×3 geändert; NF-09 (macOS-Kompatibilität) ergänzt
1.2	2026-04-21	Tic-Tac-Toe gemäß Lastenheft (LF4020) auf 6×6 / 4 in einer Reihe zurückgesetzt; PF-M03, Abschnitt 4.2, Bewertungsfunktion, TC-09–TC-11 wiederhergestellt
1.3	2026-04-22	Abgleich mit Lastenheft: PF-K03 (Dame) und PF-K04 (alternative KI-Schnittstelle) als Kann-Ziele ergänzt; neuer Abschnitt 12 mit Dokumentations- und Konzeptanforderungen (DOC-01–DOC-13) aus LD5100–LD5500; Abschnitt 14.2 Offene Punkte erweitert; Liefergegenstände mit Anforderungsreferenzen verknüpft
1.4	2026-04-22	Neuer Abschnitt 15: Design und Corporate Identity – Farbpalette mit Begründungen, Typografie, Designprinzipien und ASCII-Mockups aller sechs Hauptscreens (Login, Menü, Bauernschach, Tic-Tac-Toe, Ergebnis-Overlay, Bestenliste)
1.5	2026-04-22	Abschnitt 15 auf offizielles HHBKTendo Design System (Investor Edition) aktualisiert: Farbpalette auf 9 PDF-Tokens (Deep Navy, Card Navy, Mid Navy, Hot Pink, Royal Purple, Cyber Cyan, White, Slate Blue, Muted) umgestellt; Typografie von Segoe UI auf Orbitron (Display) + Exo 2 (Body) umgestellt; neue Abschnitte 15.5 Design Tokens (Radius/Spacing), 15.6 Glow & Effekte, 15.7 Python Code-Export ergänzt; alle Mockup-Farbnotationen aktualisiert

Pflichtenheft erstellt auf Basis des Lastenhefts Strategiespiele V13a, HHBK Tendo Research Center, 2026.